

UE25CS251A
DIGITAL LOGIC AND COMPUTER DESIGN

TEAM DLCD

Department of CSE and CSE (AI & ML)

DIGITAL LOGIC AND COMPUTER DESIGN

ARCHITECTURE-Parallelism

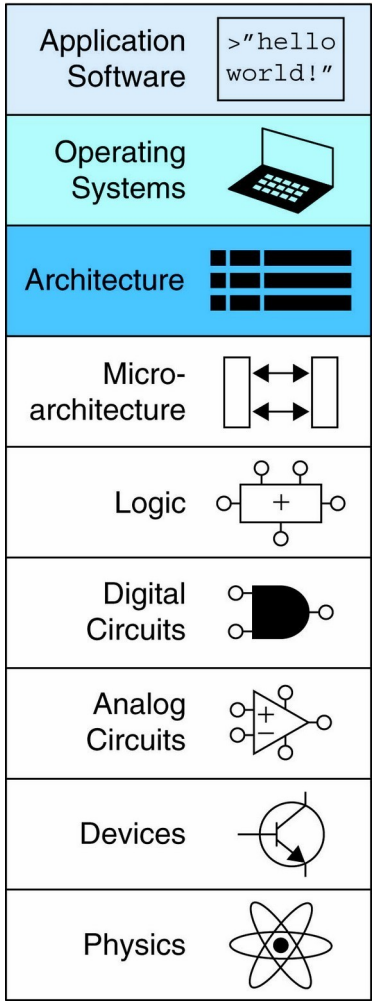
TEAM DLCD

Department of CSE and CSE (AI & ML)

DIGITAL LOGIC & COMPUTER DESIGN

TOPICS

- **Parallelism**



© Elsevier, Digital Design and Computer Architecture, 2nd

DIGITAL LOGIC & COMPUTER DESIGN

Parallelism

Parallelism is the technique of performing **multiple operations simultaneously** to improve the overall performance of a computer system. Instead of processing one task completely before starting the next, several tasks are executed at the same time using additional hardware resources or by overlapping different stages of execution.

The primary objective of parallelism is to **increase system throughput** without significantly affecting the latency of an individual task.

- Executes multiple tasks simultaneously.
- Improves processor performance.
- Makes better use of available hardware resources.
- Widely used in modern processors, GPUs, and multicore systems.

Definition: Parallelism is the simultaneous execution of multiple tasks to increase the amount of work completed in a given period.

© Elsevier, Digital Design and Computer Architecture, 2nd

DIGITAL LOGIC & COMPUTER DESIGN

Why Parallelism?

Modern computer systems must process a large number of tasks efficiently. Simply increasing the processor clock frequency is no longer sufficient because it leads to higher power consumption and heat generation.

Parallelism improves performance by executing multiple tasks simultaneously or by overlapping their execution.

Benefits

- Increases overall system throughput.
- Makes efficient use of available hardware.
- Enables higher performance without relying only on faster clocks.
- Widely used in multicore processors and GPUs.

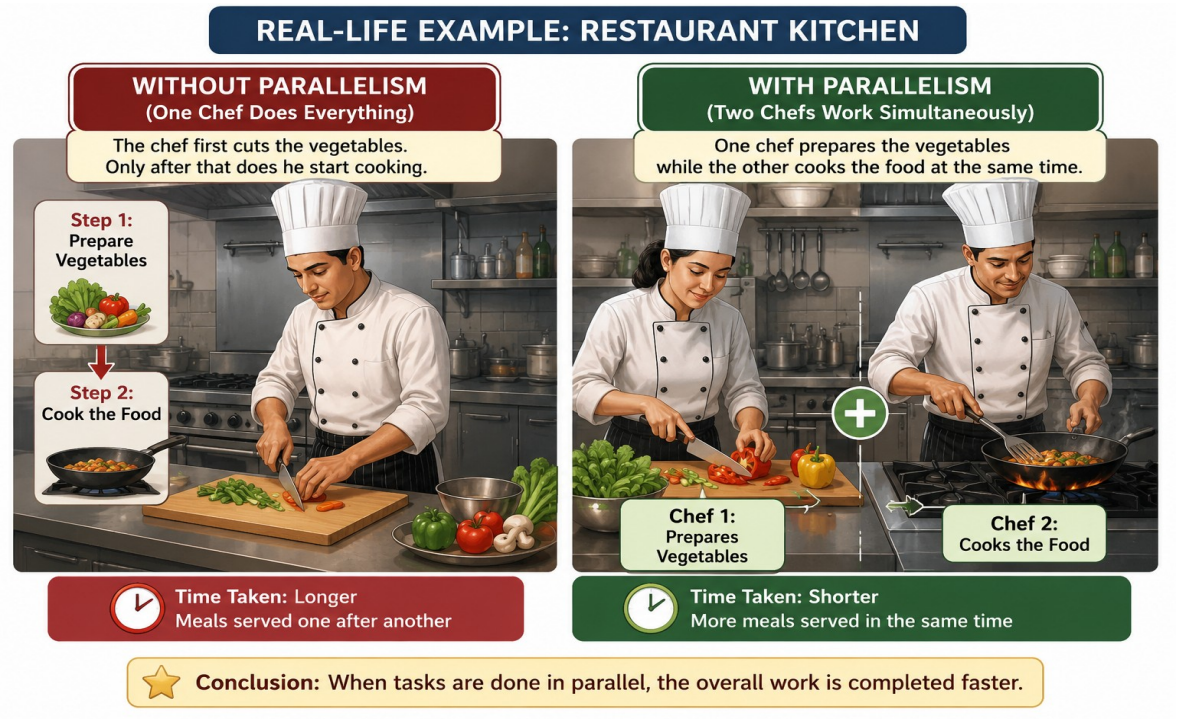
Key Point: Parallelism mainly improves throughput rather than reducing latency.

© Elsevier, Digital Design and Computer Architecture, 2nd

DIGITAL LOGIC & COMPUTER DESIGN

Parallelism

Example: Imagine a restaurant kitchen where one chef prepares vegetables while another cooks food. Since both tasks occur simultaneously, meals are prepared faster than if only one chef performed every task.



© Elsevier, Digital Design and Computer Architecture, 2nd

DIGITAL LOGIC & COMPUTER DESIGN

Performance Metrics – Latency and Throughput

Before studying parallelism, it is important to understand two performance metrics used to evaluate a computer system.

1. Latency

Latency is the **time required for a single task (token) to travel from input to output**.

It measures how long one operation takes to complete.

Formula

Latency=Time required for one task

2. Throughput

Throughput is the **number of tasks completed per unit time**.

Higher throughput means the processor finishes more work every second.

© Elsevier, Digital Design and Computer Architecture, 2nd

DIGITAL LOGIC & COMPUTER DESIGN

Performance Metrics – Latency and Throughput

Formula

$$\text{Throughput} = \text{Number of completed tasks} / \text{Unit Time}$$

For a non-parallel system,

$$\text{Throughput} = 1 / \text{Latency}$$

Latency vs Throughput

- **Latency** is the time taken to complete a single task from start to finish. Lower latency means faster response for an individual task.
- **Throughput** is the number of tasks completed in a given amount of time. Higher throughput means the system can process more work overall.

Key Point: A system can achieve **high throughput** by processing many tasks simultaneously, but the time taken for each individual task (latency) may still remain the same, or even increase. Therefore, improving throughput does **not** always reduce latency.

© Elsevier, Digital Design and Computer Architecture, 2nd

DIGITAL LOGIC & COMPUTER DESIGN

Example – Cookie Throughput and Latency (Textbook Numerical)

Problem: Ben is baking cookies.

- Rolling cookies = **5 minutes**
- Baking cookies = **15 minutes**

After one tray finishes baking, only then does he start preparing the next tray. Find latency and throughput

Solution: Step 1: Latency

Total time for one tray

5+15=20 minutes =1/3 hour

Therefore, Latency=20 minutes

© Elsevier, Digital Design and Computer Architecture, 2nd

Example – Cookie Throughput and Latency (Textbook Numerical)

Step 2: Throughput

One tray is completed every 20 minutes.

In one hour,

60/20=3

Hence,

Throughput=3 trays / hour

Observation:

Although each tray requires 20 minutes, only three trays can be produced every hour because no tasks overlap.

© Elsevier, Digital Design and Computer Architecture, 2nd

DIGITAL LOGIC & COMPUTER DESIGN

Types of Parallelism

Parallelism is broadly classified into **two categories**.

1. Spatial Parallelism

Spatial parallelism improves performance by **adding multiple copies of hardware**. Each hardware unit performs a different task simultaneously.

Examples:

- Multiple processors
- Multiple ALUs
- Multicore CPUs

Advantages:

- Higher throughput
- True simultaneous execution

Disadvantage:

- Higher hardware cost

© Elsevier, Digital Design and Computer Architecture, 2nd

Types of Parallelism

2. Temporal Parallelism (Pipelining)

Temporal parallelism divides one task into multiple stages.

Different tasks occupy different stages simultaneously, just like an assembly line.

Example

Instruction Fetch -> Instruction Decode -> Execute -> Memory -> Write Back

Advantages

- Better hardware utilization
- Lower hardware cost than spatial parallelism

© Elsevier, Digital Design and Computer Architecture, 2nd

Types of Parallelism

Spatial Parallelism

- Executes multiple operations simultaneously by **duplicating hardware resources**.
- Requires **additional hardware**, such as multiple processors or multiple functional units.
- Provides high performance but comes with **higher implementation cost**.

Temporal Parallelism

- Improves performance by **dividing a task into sequential pipeline stages**.
- Reuses the **same hardware** across different stages over time.
- Requires less additional hardware, making it **more cost-effective**.
- Commonly implemented using **pipelining** in processors.

Key Point: Spatial parallelism increases performance by adding more hardware, whereas temporal parallelism increases performance by overlapping different stages of execution using the same hardware.

© Elsevier, Digital Design and Computer Architecture, 2nd

DIGITAL LOGIC & COMPUTER DESIGN

Cookie Parallelism (Textbook Numerical)

Ben wants to bake cookies faster.

Case 1: Spatial Parallelism

Ben asks Alyssa to help. Now, Both bake simultaneously.

Latency = 20 minutes

Each person produces : 3 trays/hour

Therefore, Throughput = 3+3 = 6 = 6 trays/hour

Case 2: Temporal Parallelism

Ben buys another tray. While Tray 1 is baking, he prepares Tray 2. Still, latency = 20 minutes

One tray enters oven every 15 minutes

Therefore, Throughput = 60 / 15=4 trays/hour

© Elsevier, Digital Design and Computer Architecture, 2nd

DIGITAL LOGIC & COMPUTER DESIGN

Cookie Parallelism (Textbook Numerical)

Case 3: Spatial + Temporal

Two people. Two trays. Result,

Latency = 20 minutes

Throughput = 8 trays/hour

Conclusion:

Parallelism improves **throughput** but **does not reduce latency**.

© Elsevier, Digital Design and Computer Architecture, 2nd

DIGITAL LOGIC & COMPUTER DESIGN

Mathematical Model of Parallelism

Consider a task having latency **L**.

Without Parallelism

Throughput=1 / L

Spatial Parallelism

Suppose N hardware units exist. Then

Throughput=N / L

Temporal Parallelism

Suppose, Task divided into N equal stages. Then

Throughput=N / L

© Elsevier, Digital Design and Computer Architecture, 2nd

DIGITAL LOGIC & COMPUTER DESIGN

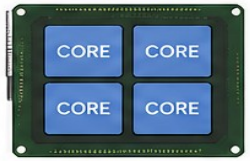
Mathematical Model of Parallelism

Real Processor Examples

SPATIAL PARALLELISM

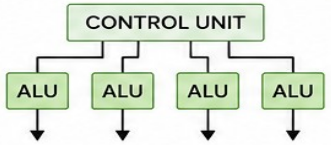
Multiple operations occur at the same time using multiple hardware units.

1. Multicore CPUs



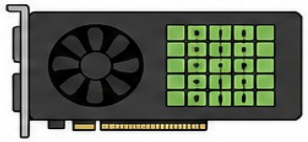
- Multiple cores execute different instructions simultaneously.
- Example: Intel Core i7, AMD Ryzen

2. Multiple ALUs



- Several ALUs perform operations in parallel.
- Used in superscalar processors.

3. GPU Cores

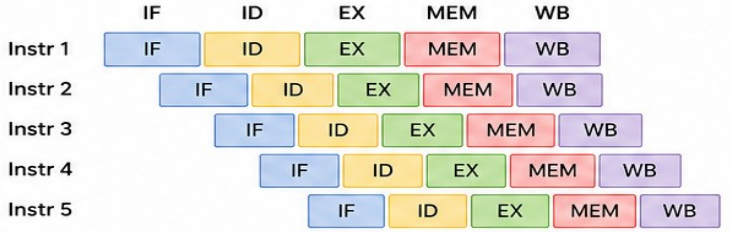


- Thousands of small cores work in parallel on large data sets.
- Example: NVIDIA RTX series

TEMPORAL PARALLELISM

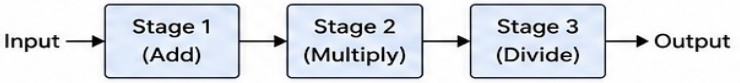
Multiple operations of the same task are overlapped in time.

1. Instruction Pipelining



- Different instructions are at different stages of execution simultaneously.

2. Arithmetic Pipelines



- Long arithmetic operations are divided into pipeline stages to increase throughput.

Modern processors combine both spatial and temporal parallelism to maximize performance and efficiency.

© Elsevier, Digital Design and Computer Architecture, 2nd

DIGITAL LOGIC & COMPUTER DESIGN

Mathematical Model of Parallelism

If stages are unequal, the longest stage determines speed. If longest stage latency is L1, then

Throughput=1 / L1

Key Observation:

Increasing the number of stages increases throughput only if the workload is evenly balanced.

© Elsevier, Digital Design and Computer Architecture, 2nd

DIGITAL LOGIC & COMPUTER DESIGN

Pipelining Example (Textbook Numerical)

Circuit Without Pipeline

Given Clock-to-Q delay is 0.3 ns, Logic blocks are 3 ns, 2 ns, 4 ns, Setup time is 0.2 ns

Step 1: Cycle Time, TC=0.3+3+2+4+0.2=9.5ns

Step 2: Latency = 9.5ns

Step 3: Throughput = 1 / 9.5ns = 105MHz

© Elsevier, Digital Design and Computer Architecture, 2nd

DIGITAL LOGIC & COMPUTER DESIGN

Two-Stage Pipeline (Textbook Numerical)

Now insert one register after Block 3.

Stage 1: 0.3+3+2+0.2 = 5.5ns

Stage 2: 0.3+4+0.2 = 4.5ns

Clock period equals longest stage.

Therefore, TC = 5.5ns

Throughput = 1 / 5.5ns = 182MHz

Performance Comparison

Without Pipelining

- **Clock Cycle Time: 9.5 ns**
- **Maximum Throughput: 105 MHz**
- The entire task is completed in a single long cycle, resulting in a lower operating frequency.

© Elsevier, Digital Design and Computer Architecture, 2nd

DIGITAL LOGIC & COMPUTER DESIGN

Two-Stage Pipeline (Textbook Numerical)

With Two-Stage Pipelining

- **Clock Cycle Time: 5.5 ns**
- **Maximum Throughput: 182 MHz**
- Dividing the circuit into two stages reduces the critical path delay, allowing the processor to run at a higher clock frequency.

Observation:

By introducing pipelining, the clock cycle time decreases from **9.5 ns** to **5.5 ns**, increasing the maximum operating frequency from **105 MHz** to **182 MHz**. This significantly improves **throughput**, while the latency of an individual task remains nearly unchanged.

© Elsevier, Digital Design and Computer Architecture, 2nd

Why Pipeline is Attractive

Pipelining is one of the most effective forms of temporal parallelism.

Advantages

- Does not require duplicate hardware.
- Divides long combinational logic into shorter stages.
- Enables a shorter clock period.
- Improves throughput significantly.
- Makes efficient use of hardware resources.

Important Note

Pipeline registers isolate consecutive stages, preventing one token from interfering with another.

© Elsevier, Digital Design and Computer Architecture, 2nd

DIGITAL LOGIC & COMPUTER DESIGN

Advantages, Limitations & Applications

Advantages

- Increases throughput.
- Improves processor utilization.
- Enables higher clock frequencies.
- Forms the basis of modern pipelined processors.
- Essential in multicore CPUs and GPUs.

Limitations

- Spatial parallelism requires additional hardware.
- Pipelining introduces pipeline registers and control complexity.
- Unequal pipeline stages reduce efficiency.
- Data hazards and control hazards may reduce performance.

© Elsevier, Digital Design and Computer Architecture, 2nd

DIGITAL LOGIC & COMPUTER DESIGN

Advantages, Limitations & Applications

Applications

- Multicore processors
- Graphics Processing Units (GPUs)
- Supercomputers
- Digital Signal Processors (DSPs)
- Machine Learning accelerators

Key Takeaways

- **Latency** measures the time taken to complete one task.
- **Throughput** measures the number of tasks completed per unit time.
- **Spatial Parallelism** uses multiple hardware units.
- **Temporal Parallelism** overlaps task execution through pipelining.
- Parallelism mainly improves **throughput**, while latency often remains unchanged.
- Pipelining is one of the most effective and widely used forms of temporal parallelism in modern computer architecture.

© Elsevier, Digital Design and Computer Architecture, 2nd

Pipeline Hazards

Pipeline hazards are conditions that prevent instructions from executing smoothly through the pipeline. They introduce **stalls (bubbles)**, reducing the overall throughput.

1. Data Hazard

- Occurs when an instruction requires data that is still being produced by a previous instruction.
- The dependent instruction must wait until the data is available.

2. Control Hazard

- Caused by branch or jump instructions.
- The processor may fetch incorrect instructions until the branch decision is known.

3. Structural Hazard

- Occurs when multiple instructions need the same hardware resource simultaneously.
- One instruction must wait until the resource becomes available.

© Elsevier, Digital Design and Computer Architecture, 2nd

DIGITAL LOGIC & COMPUTER DESIGN

Advantages, Limitations & Applications

Question:

The circuit in the textbook is divided into a **three-stage pipeline**. The three pipeline stages have delays of **3.5 ns, 2.5 ns, and 4.5 ns**, respectively. Determine:

1. The clock cycle time
2. The latency of the circuit
3. The throughput

Answer

The clock must be slow enough for the **slowest pipeline stage**.

Tc=max(3.5,2.5,4.5); Tc=4.5 ns

Latency:

There are 3 pipeline stages.

Latency = 3Tc = 3(4.5)=13.5 ns

© Elsevier, Digital Design and Computer Architecture, 2nd

Advantages, Limitations & Applications

Throughput:

Throughput = 1/Tc
= 14.5×10^-9
≈ 222 MHz

Final Answer:

- **Cycle time = 4.5 ns**
- **Latency = 13.5 ns**
- **Throughput = 222 MHz**

© Elsevier, Digital Design and Computer Architecture, 2nd

DIGITAL LOGIC & COMPUTER DESIGN

MCQ

Parallelism primarily aims to improve:

A) Memory capacity
B) Throughput
C) Register size
D) Cache size

© Elsevier, Digital Design and Computer Architecture, 2nd

DIGITAL LOGIC & COMPUTER DESIGN

MCQ

A token in a parallel system refers to:

A) A processor core

B) A unit of work moving through the system

C) A clock pulse

D) A memory address

© Elsevier, Digital Design and Computer Architecture, 2nd

DIGITAL LOGIC & COMPUTER DESIGN

MCQ

Which type of parallelism increases performance by using multiple hardware units?

A) Temporal Parallelism
B) Spatial Parallelism
C) Instruction Parallelism
D) Data Compression

© Elsevier, Digital Design and Computer Architecture, 2nd

DIGITAL LOGIC & COMPUTER DESIGN

MCQ

Pipeline registers are inserted to:

A) Increase memory size
B) Divide combinational logic into stages
C) Reduce cache misses
D) Store operating system data

© Elsevier, Digital Design and Computer Architecture, 2nd

DIGITAL LOGIC & COMPUTER DESIGN

MCQ

Modern processors achieve high performance by combining:

A) Only higher clock frequencies
B) Only larger memories
C) Spatial and temporal parallelism
D) Only single-cycle execution

© Elsevier, Digital Design and Computer Architecture, 2nd

DIGITAL LOGIC & COMPUTER DESIGN

MCQ

Increasing the number of pipeline stages generally:

A) Improves throughput but may increase latency
B) Reduces throughput significantly
C) Eliminates pipeline hazards completely
D) Makes the processor slower in all cases

© Elsevier, Digital Design and Computer Architecture, 2nd

DIGITAL LOGIC & COMPUTER DESIGN

MCQ

Which metric indicates how much useful work a system completes over time?

A) Latency
B) Throughput
C) Clock Period
D) Register Width

© Elsevier, Digital Design and Computer Architecture, 2nd

DIGITAL LOGIC & COMPUTER DESIGN

MCQ

Which of the following is a real-world example of temporal parallelism?

A) Two cashiers serving customers simultaneously

B) An assembly line where different workers perform different stages on different products

C) Two printers printing the same document

D) Two CPUs installed on a motherboard without coordination

© Elsevier, Digital Design and Computer Architecture, 2nd

DIGITAL LOGIC & COMPUTER DESIGN

MCQ

Answer Key

Q1: B **Q2: B** **Q3: B** **Q4: B** **Q5: C** **Q6: B** **Q7: B** **Q8: B**



THANK YOU

Team DLCD

Department of CSE and CSE(AI & ML)